CULINARY GRAPH

Deployment URL: https://culinary.page/
Repository URL: https://github.com/yaseminyumak/SWE-573
Release Version URL: https://github.com/yaseminyumak/SWE-573/releases/tag/v0.9

Course: SWE 573
Instructor: Suzan Üsküdarlı

HONOR CODE
Related to the submission of all the project deliverables for the Swe573 Spring 2026 semester project reported in this report, I, Yasemin Yumak declare that:
- I am a student in the Software Engineering MS program at Bogazici University and am registered for Swe573 course during the Spring 2026 semester.
- All the material that I am submitting related to my project (including but not limited to the project repository, the final project report, and supplementary documents) have been exclusively prepared by myself.
- I have prepared this material individually without the assistance of anyone else with the exception of permitted peer assistance, which I have explicitly disclosed in this report.

Name Surname: Yasemin Yumak
Student Id: 2024719177
Date: 10/05/2026

Table of Contents

1. Overview

This report presents Culinary Graph, a web application developed for SWE-573 that supports region-aware culinary knowledge: contributors document techniques, ingredients, and recipes, while guests can browse, search, and explore structured content. The work follows a requirements-driven lifecycle from covering analysis, design (including UML artifacts and UI mockups), implementation, testing, and deployment. The report summarizes the problem context, functional and non-functional requirements aligned with the SRS, the chosen architecture and key features, and how outcomes were verified.

Culinary Graph is a web platform for sharing cooking knowledge with a clear focus on where food comes from and why it matters culturally. Instead of isolated recipes, it brings together recipes, cooking techniques, and ingredients in one place, with room for stories, regional notes, and practical details such as substitutions or seasonality. The aim is to help learners, curious cooks, and anyone interested in food heritage find trustworthy, structured information—and to let contributors add their own knowledge in a consistent way. The project was built as the final coursework product for SWE-573: a working application you can open in a browser, sign in to, and use end-to-end—not only diagrams or documents. Users can explore content anonymously, then register and log in to create and edit their own entries. The same application can be run on a developer machine or deployed to a simple cloud server, so results are easy to reproduce and demonstrate. Overall, Culinary Graph shows how a small, focused food-and-heritage idea can become a real, usable system: centered on region-aware culinary knowledge, designed for clarity for readers, and for a smooth contribution flow for authors.

2. AI Use Declaration

Use of AI-assisted tools is disclosed for academic transparency.

2.1. Documentation and project reports

Tool: Cursor (Composer / agent-style assistance)

Used for: Drafting and revising Markdown under reports/ and small README cross-links—for example expanding ProjectPlan.md (phased timeline, unified work-breakdown table with BL/DG IDs, milestones), restructuring USER_MANUAL.md (flows, deployed URL, feature descriptions), and aligning deliverable indexes with the SRS.

Reliance: Partial to major generation of prose and tables; factual claims and dates were checked against the real product and calendar.

Validation: Manual read-through; link and path checks; edits applied after review so the text matched implemented behavior and assignment wording.

2.2. Diagrams (software design)

Tool: Cursor

Used for: Authoring and iterating class_diagram.mermaid (Mermaid class diagram: entities, relationships, readability); earlier exploratory PlantUML wording in chat was not kept as a separate committed artifact—the Mermaid source in reports/ is the submitted form.

Reliance: Partial generation (structure and labels from the codebase); several revision passes for Mermaid Live compatibility and layout.

Validation: Compared to JPA entities and APIs in culinarygraph-backend; diagram rendered in Mermaid Live / editor preview; inconsistencies (e.g. reserved words, cardinality) fixed manually.

2.3. Source code

Tool: Claude

Used for: Assistance during implementation of the Spring Boot backend (culinarygraph-backend/) and the React frontend (culinarygraph-frontend/)—including scaffolding, endpoints, UI components, refactors, and bug fixes as the feature set grew.

Reliance: Partial to major generation depending on the file or feature; the author integrated suggestions, renamed symbols to match project conventions, and trimmed scope where needed.

Validation: Local builds (mvn, frontend bundler), manual exercise of main flows (auth, catalog CRUD, recipes, social), and code review before commits; server-side rules (e.g. ownership) were verified against the SRS and Keycloak behavior.

2.4. Test and synthetic data

Tool: Claude Opus 4.6

Used for: Drafting and refining synthetic seed content used to populate the app for demos and manual testing—for example scripted or structured content in docker/seed-content.sh (and related data shaped for Postgres/REST).

Reliance: Major generation of example names, descriptions, and relationships; treated as non-production fiction only.

Validation: No real personal data; scripts run against local/docker DB; spot-checks in the UI that seeded entities list, filter, and link correctly; edits applied where tone or fields did not match the schema.

2.5. What was not AI-generated

- UI mockups under reports/mock-ups/ were produced in Balsamiq (human design), not by generative AI.

3. Test Account

For testing purposes, the deployed system is accessible through the provided deployment URL. The application supports both guest and authenticated contributor access. Guest users can browse recipes, techniques, ingredients, use the global search functionality, and view likes, comments, and contributor information without authentication.

To test contributor-specific features such as creating, editing, and deleting content, adding comments and likes, and managing images, a dedicated test contributor account is provided below:

Contributor Test Account
Username: suzan.uskudarli
Email: uskudarli@gmail.com
Password: Test123!

This account can be used to test all role-based functionalities implemented in the system.

4. Software Requirements Specification

4.1. System Features (Functional Requirements)

4.1.1. Identity and Access

- FR-1 — The system shall allow a Guest to register for an account (e.g. using email and password or other defined identity providers).
- FR-2 — The system shall allow a registered user to log in and log out, and shall maintain a session for an authenticated user.
- FR-3 — The system shall enforce role-based access: Guest (browse, search where implemented, register, and view techniques, ingredients, and recipes), Contributor (add, edit, and delete their own techniques, ingredient profiles, and recipes, and use relationship or association controls provided by the forms).

4.1.2. Techniques

- FR-4 — The system shall allow a Contributor to add a technique with at least: name, steps (ordered instructions), photos (optional images attached to the technique), country, region, cultural or contextual notes, optional prerequisites, and optional related ingredients (linked to catalog ingredient entries where the feature is supported).
- FR-5 — The system shall allow users to search and filter techniques (e.g. by text, region, country).
- FR-6 — The system shall provide a technique detail view showing all stored fields for the technique and related ingredients where recorded.
- FR-7 — The system shall allow a Contributor to edit and delete only the techniques they have created.

4.1.3. Ingredients

- FR-8 — The system shall allow a Contributor to add an ingredient profile with at least: name, country, region(s), seasonality, traditional sourcing or usage notes, culturally appropriate substitutions, and optional related techniques (linked to catalog technique entries where the feature is supported).
- FR-9 — The system shall allow users to search and filter ingredient profiles (e.g. by text, region, or season).
- FR-10 — The system shall provide an ingredient detail view showing all stored fields and related techniques.
- FR-11 — The system shall allow a Contributor to edit and delete only the ingredient content they have created.

4.1.4. Recipes

- FR-12 — The system shall allow a Contributor to add a recipe with at least: title, description, difficulty, duration, ordered preparation steps, and a list of ingredients; for each ingredient line the system shall support optional substitution notes. The system shall support optional tags, optional photos, and an optional origin or provenance story.
- FR-13 — The system shall allow users to search and filter recipes (e.g. by text, tags, difficulty, or other defined criteria).
- FR-14 — The system shall provide a recipe detail view showing stored fields (including steps, ingredients, optional tags, photos, and origin story where provided).
- FR-15 — The system shall allow a Contributor to edit and delete only the recipes they have created.
- FR-16 — From a recipe detail view (or equivalent recipe presentation), when the user selects a linked technique name or a linked ingredient name that corresponds to catalog content, the system shall navigate to the corresponding technique or ingredient detail page.

4.1.5. Data Entry and Associations

- FR-17 — When creating or editing techniques, ingredient profiles, or recipes, the system shall allow the user to specify country (and region where applicable) and to associate content with existing catalog ingredients and existing catalog techniques where those association features are provided (e.g. pickers, multi-select, or relationship fields).

4.1.6. Discovery and Navigation

- FR-18 — The system shall provide a homepage that displays highlighted and/or recently added techniques, ingredients, and recipes.

4.1.7. Attribution and authorship

- FR-21 — The system shall display authorship or contributor identity for each technique, ingredient profile, and recipe in list and detail contexts where the information is available

(e.g. display name or username derived from the identity provider, and/or a link to the contributor's profile when a profile view exists). Guests and authenticated users shall be able to see who created the entry (and, where the product exposes it, who last updated it), consistent with the identity provider and any applicable privacy constraints.

4.1.8. Engagement (Comments and Likes)

- FR-19 — The system shall allow authenticated users to comment on techniques, ingredient profiles, and/or recipes (e.g. a comment thread or list per entity), subject to authentication and basic validation (e.g. length limits, empty comment rejection).
- FR-20 — The system shall allow authenticated users to like (or positively rate) techniques, ingredient profiles, and/or recipes; the system shall persist likes per user where duplicate likes are not allowed, and shall display an aggregated like count (or clear like state) on the relevant detail or list views.

4.2. External Interface Requirements

4.2.1. User Interfaces

- The system shall provide a web-based user interface.
- The interface shall adapt to the user's role (Guest vs Contributor), for example through role-appropriate menus and actions (e.g. Contributors see controls to add or edit their own content when signed in).

4.2.2. Hardware, Software, and Communication Interfaces

- Hardware: No specific hardware requirements beyond that needed to run a modern web browser and to host the application and database.
- Software: The client shall run in a supported web browser; the server stack is to be determined.
- Communication: The application shall use standard web protocols (e.g. HTTP/HTTPS). Use of a REST or similar API for future or external integration may be assumed at a high level; details are TBD.

4.3. Non-Functional Requirements

4.3.1. Performance

- NFR-1 — Search results shall be returned within an acceptable time (e.g. within 2–3 seconds under normal load).
- NFR-2 — Page loads shall complete within an acceptable time (e.g. under 3 seconds under normal conditions).

4.3.2. Usability

- NFR-3 — The user interface shall be designed for the target users, including home cooks who may not be familiar with technical terminology; labels and navigation shall be clear and consistent.
- NFR-4 — The application shall be usable on desktop devices with a clear, readable layout.
- NFR-5 — Search and filters (e.g. by region, season, country, or text) shall be easily discoverable and usable within the main flows (e.g. list pages or dedicated search where implemented).

4.3.3. Security and Privacy

- NFR-6 — User passwords shall be stored using a secure one-way hashing mechanism (e.g. bcrypt or Argon2); plain-text passwords shall not be stored.
- NFR-7 — Sessions shall be conducted over HTTPS where possible, and session cookies shall use secure and appropriate settings.
- NFR-8 — Only the content owner shall be permitted to edit or delete a given technique, ingredient profile, or recipe; the system shall enforce this authorization on the server.

4.3.4. Reliability and Data

- NFR-9 — User-contributed data shall be stored persistently; the system shall support or assume regular backups so that content can be recovered after a failure.
- NFR-10 — Data-modifying operations shall preserve consistency (e.g. use of transactions where multiple related updates are performed).

4.3.5. Maintainability and Compatibility

- NFR-11 — The data model and architecture shall be extensible so that new relationship types or additional fields can be added without disproportionate rework.
- NFR-12 — The web application shall support recent major versions of common desktop browsers (e.g. the last two major versions of Chrome, Firefox, Safari, Edge).

4.4. Other Requirements

4.4.1. Documentation

- User-facing documentation (e.g. how to add a technique, how to use search and filters) should be available to support adoption.
- Developer documentation (e.g. setup, architecture, API if any) should be maintained to support development and maintenance.

5. Design Documents

5.1. Use Case Diagram

The use case diagram shows the main interactions between users and the Culinary Graph system. It includes the Guest and Contributor roles and demonstrates which actions are available to each user type, such as browsing content, searching, viewing details, creating content, and managing own entries.

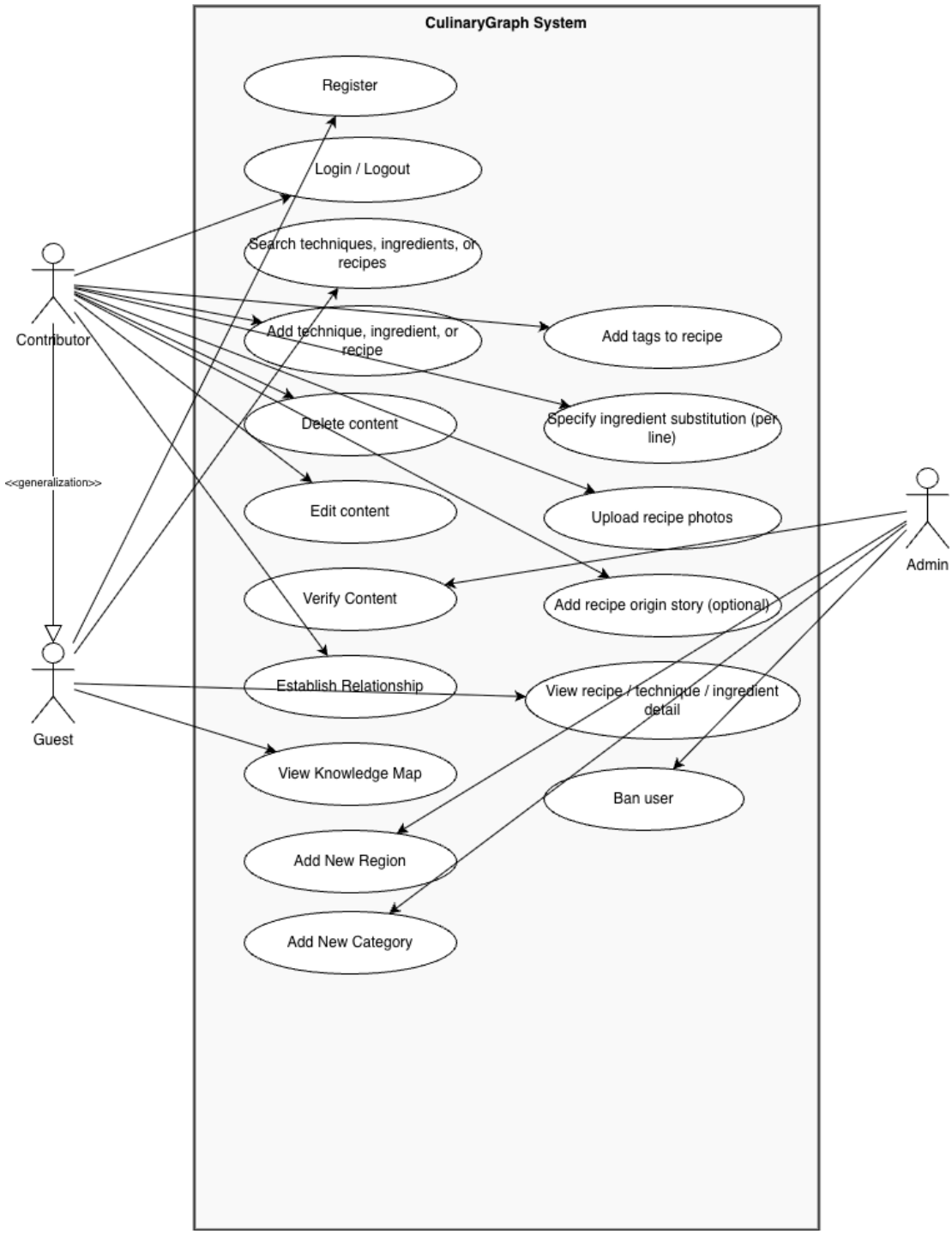

Figure 1: Use Case Diagram

5.2. Sequence Diagram

The sequence diagram illustrates the step-by-step interaction flow between the user, frontend, backend API, and database. It shows how a selected user scenario is processed by the system, including user action, request handling, validation, data storage or retrieval, and response display.

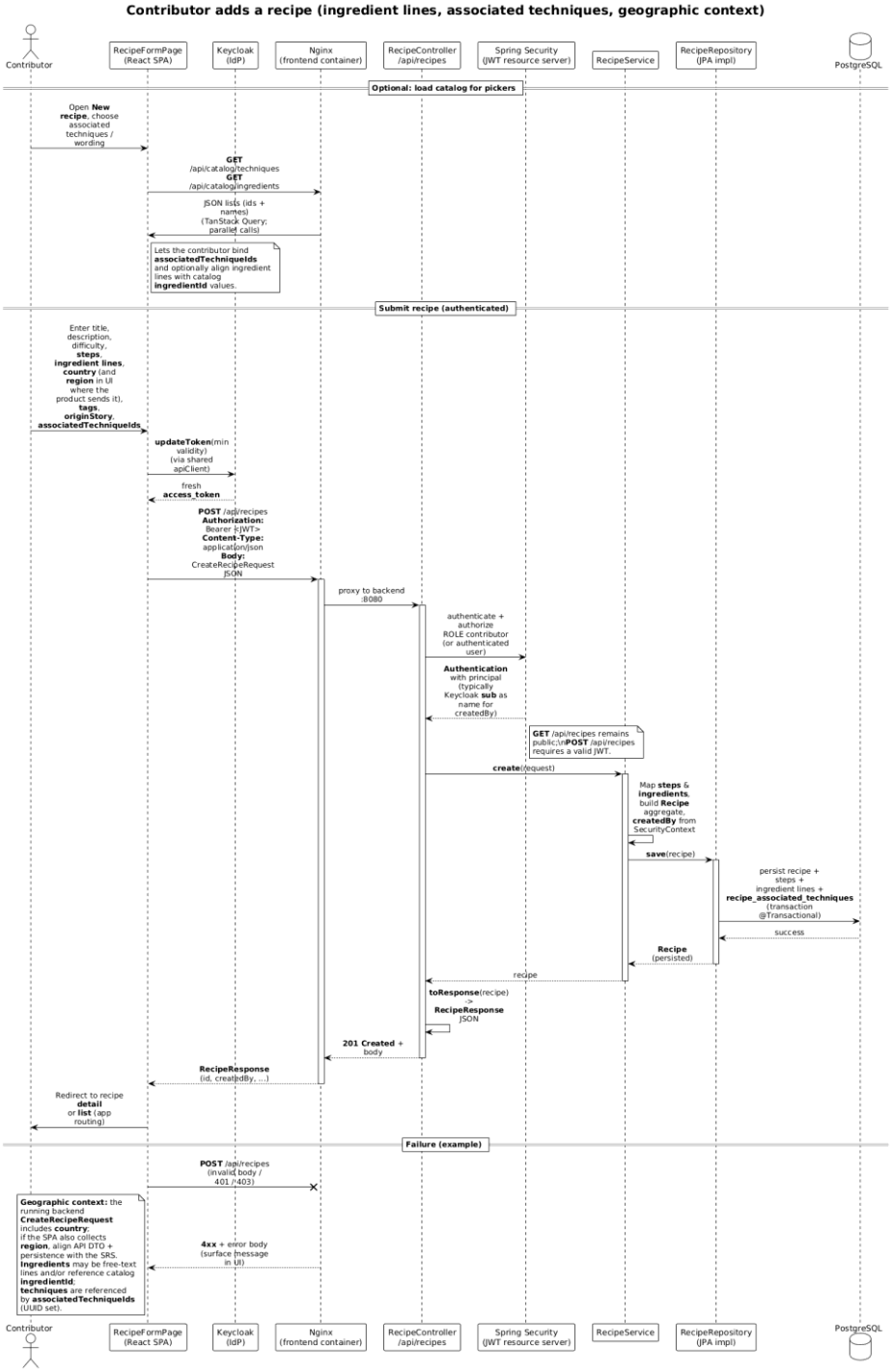

Figure 2: Sequence Diagram for adding a recipe

5.3. Class Diagram

The class diagram represents the main data structure of the Culinary Graph system. It shows key entities such as User, Recipe, Technique, Ingredient, Region, Comment, and Like, together with their attributes and relationships. It also highlights ownership and content relationships within the system.

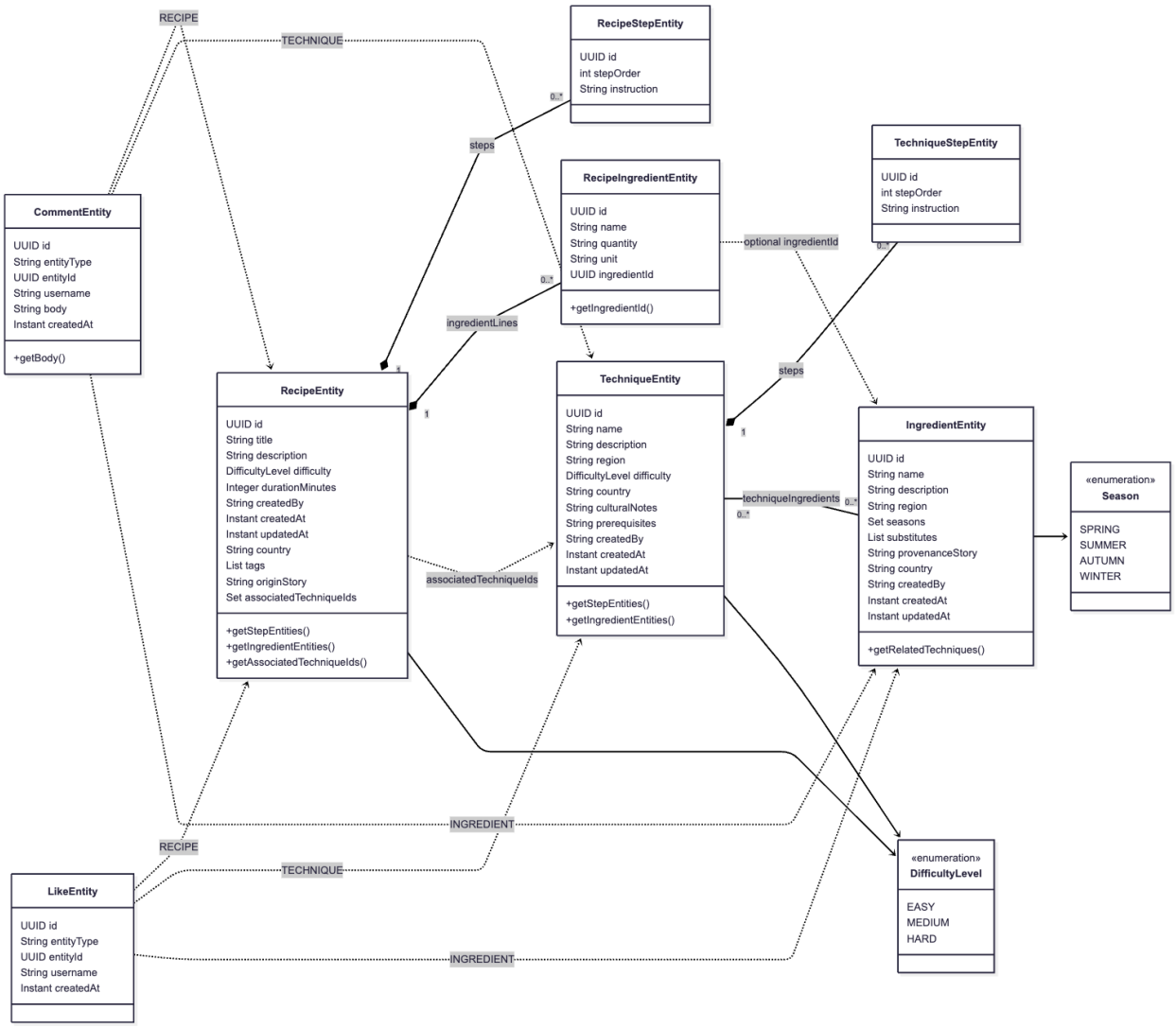

Figure 3: Class Diagram

5.4. Mock-ups

The mockups present the planned user interface of CulinaryGraph. They show how users browse, search, view, create, and interact with culinary content. The screens also demonstrate important UX decisions such as the homepage Knowledge Map, active navigation, contributor profiles, and consistent content creation forms.

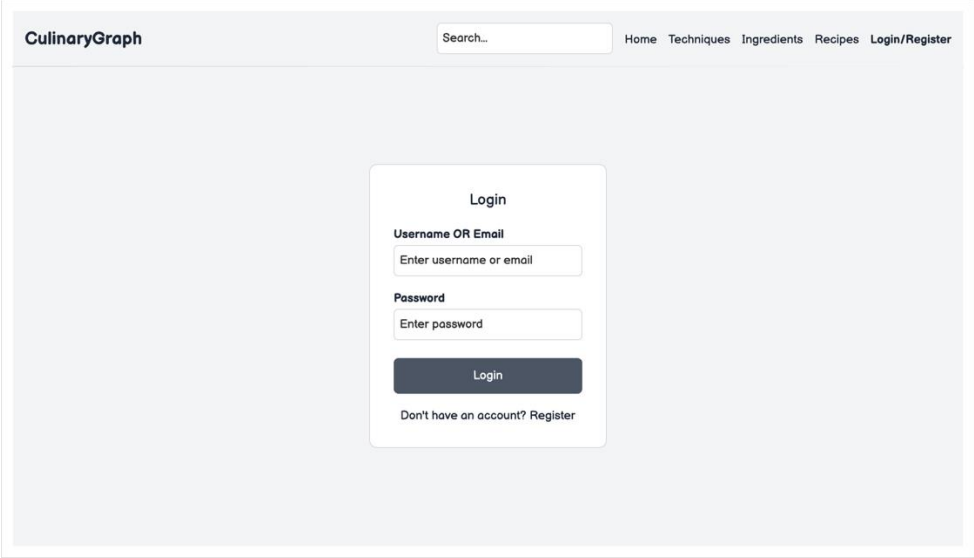

Figure 4: Log-in Page

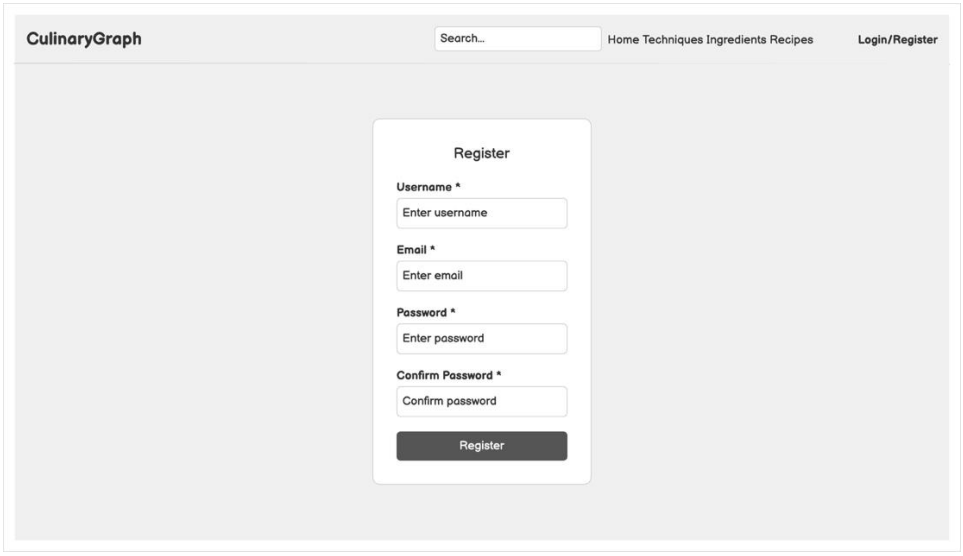

Figure 5: Register Page

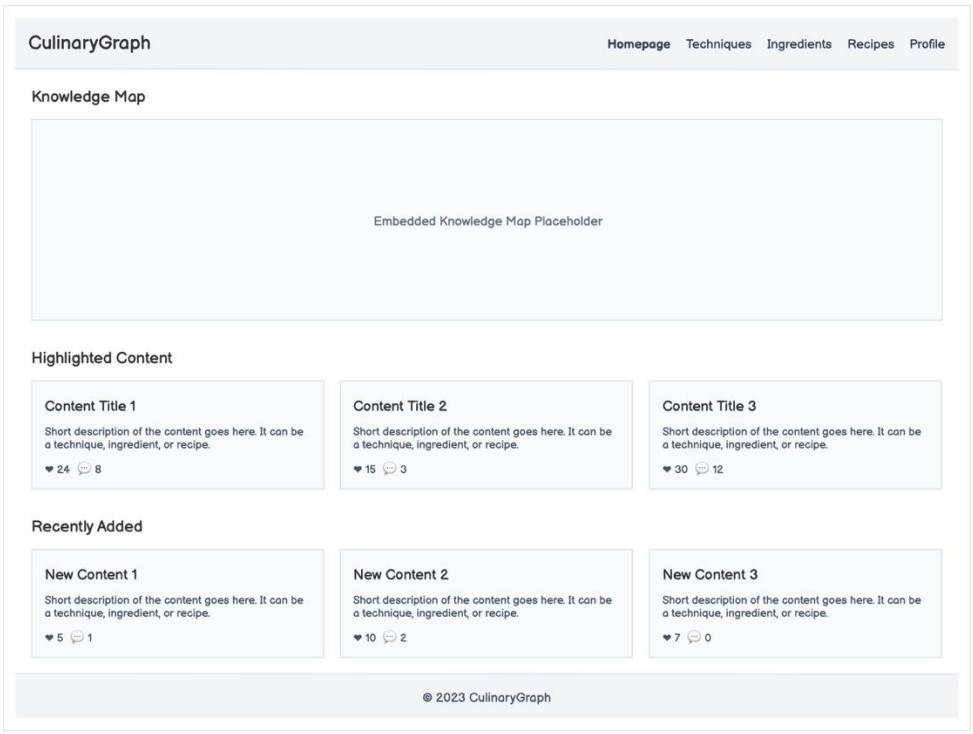

Figure 6: Home Page

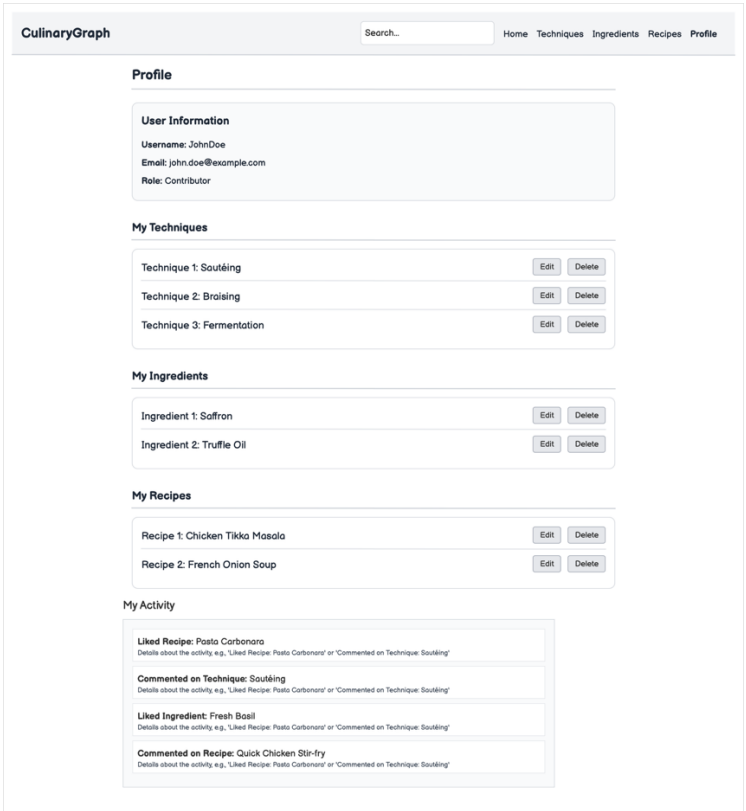

Figure 7: Profile Page

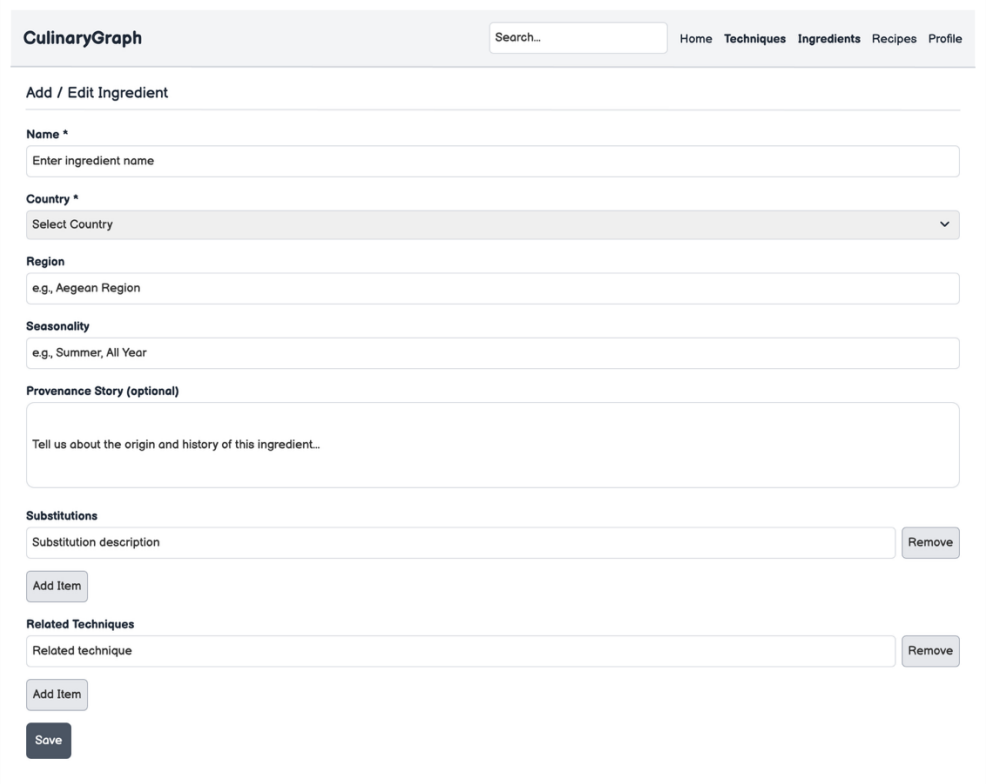

Figure 8: Add Ingredient Page

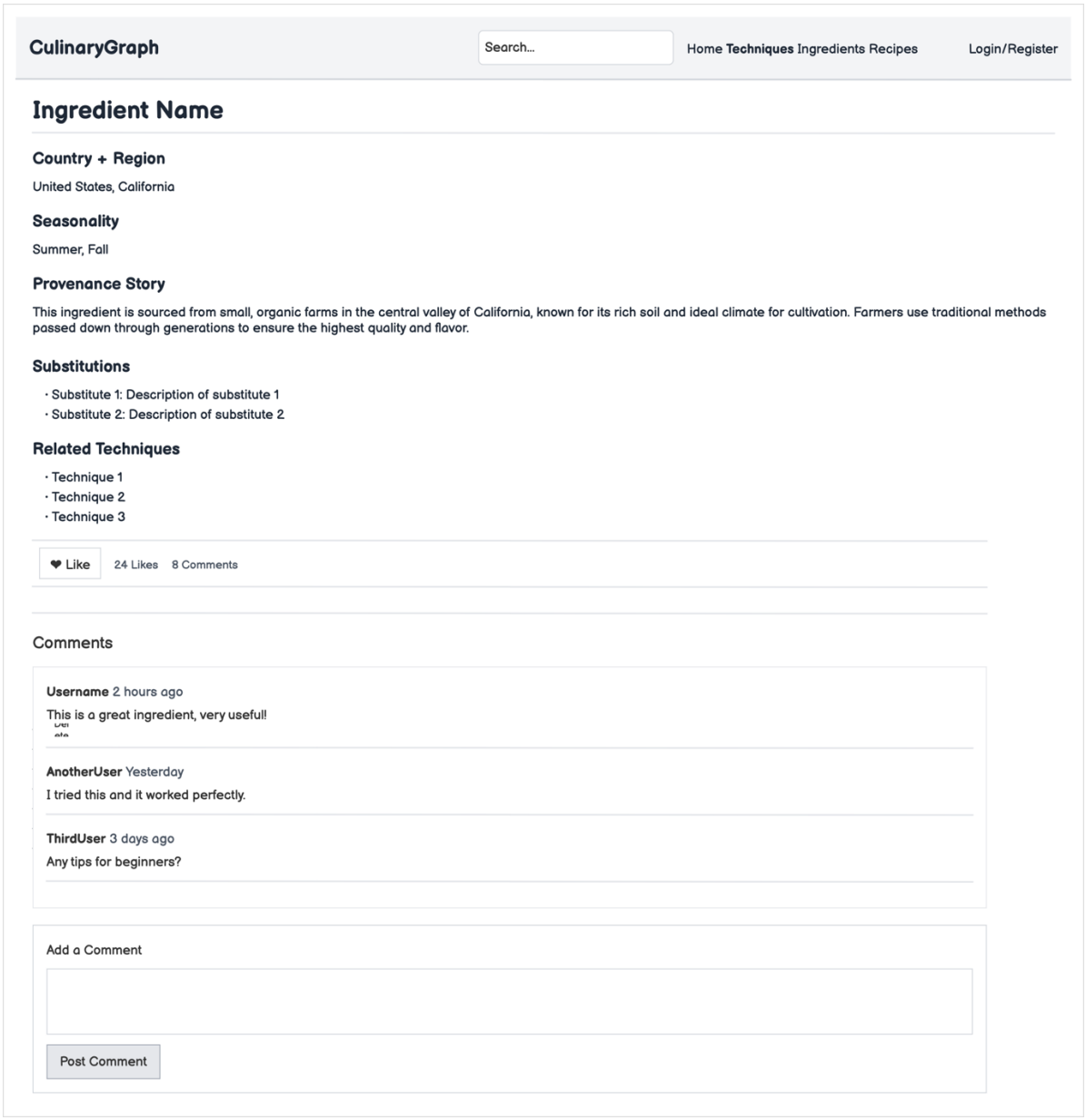

Figure 9: Ingredient Detail Page

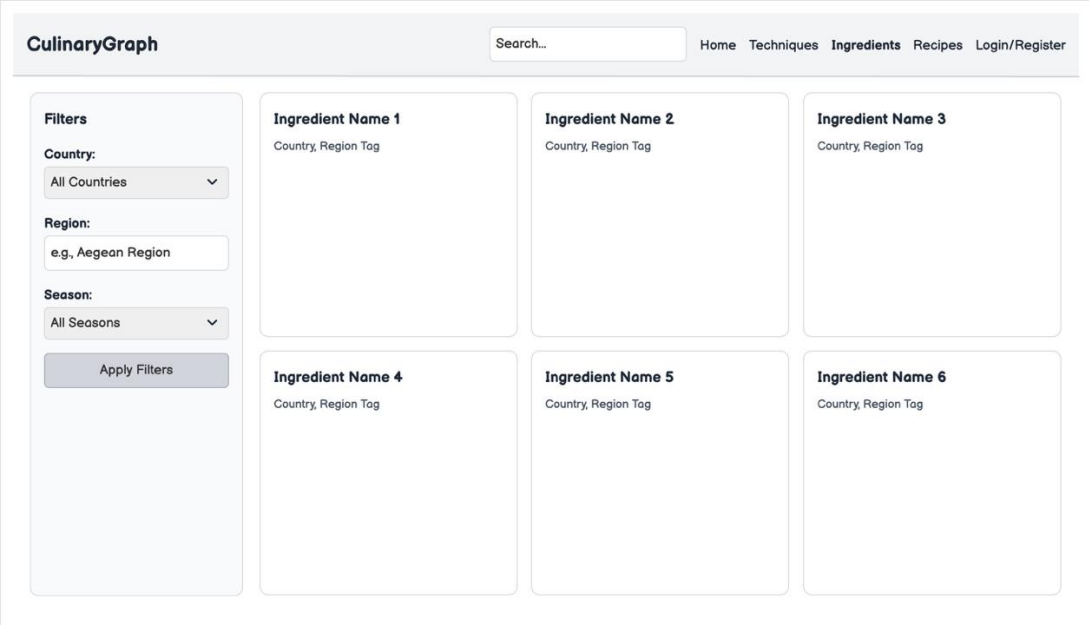

Figure 10: Ingredients List Page

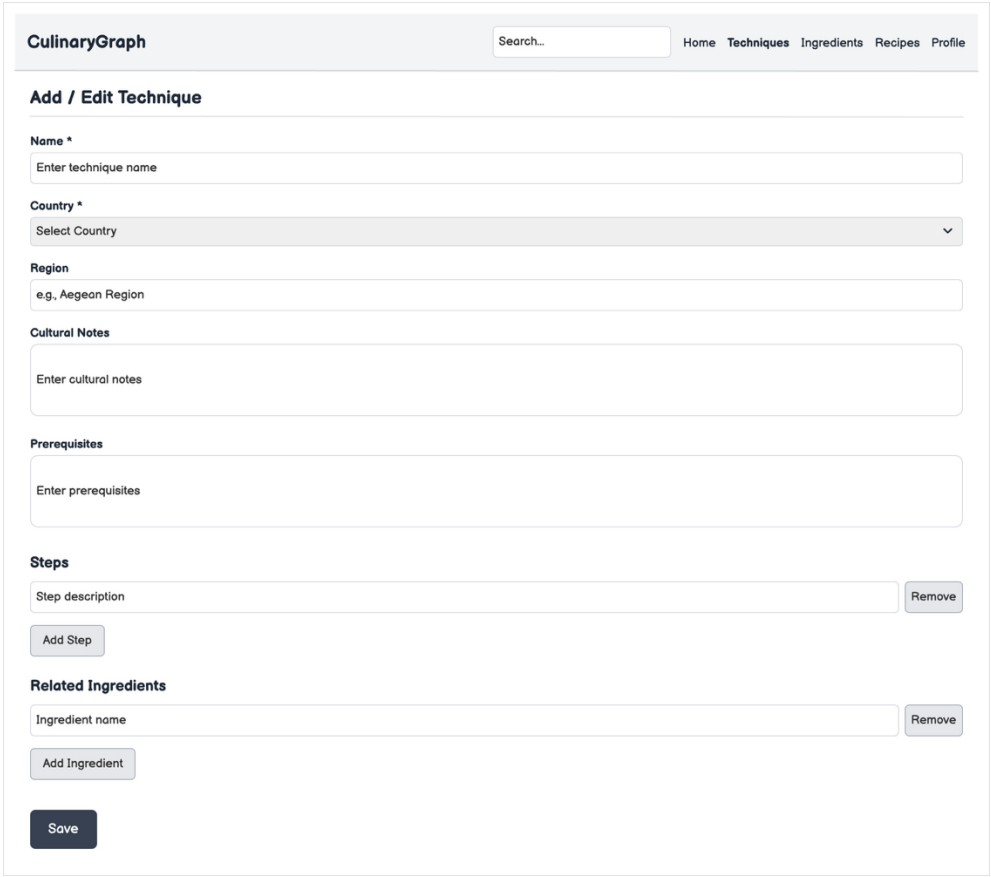

Figure 11: Add Technique Page

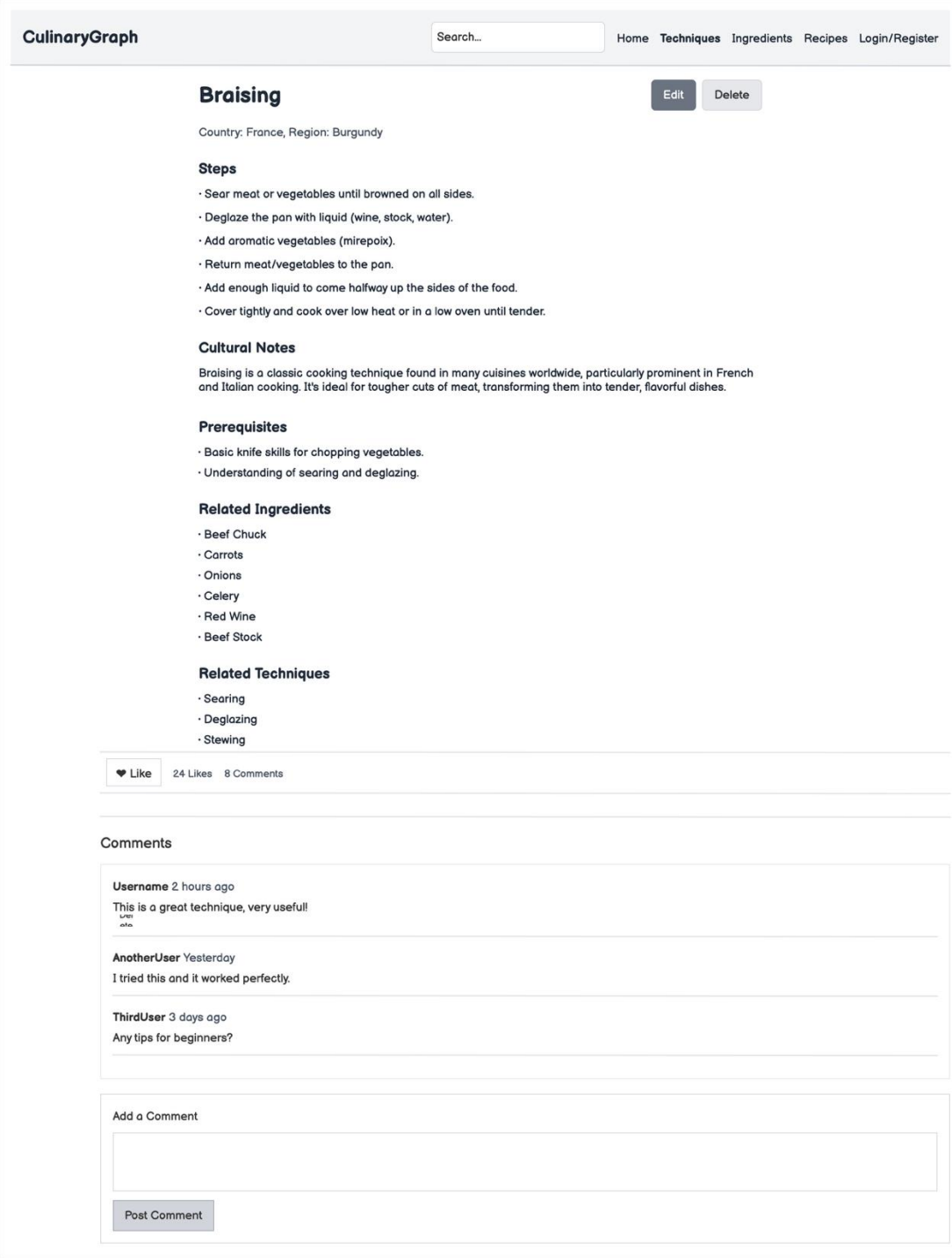

Figure 12: Technique Detail Page

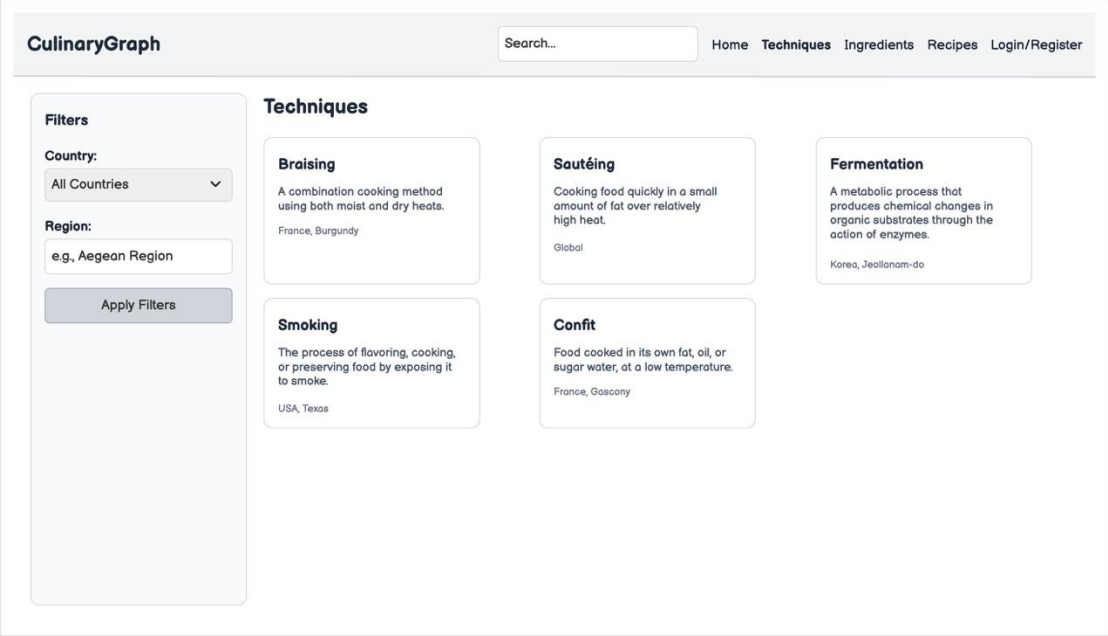

Figure 13: Technique List Page

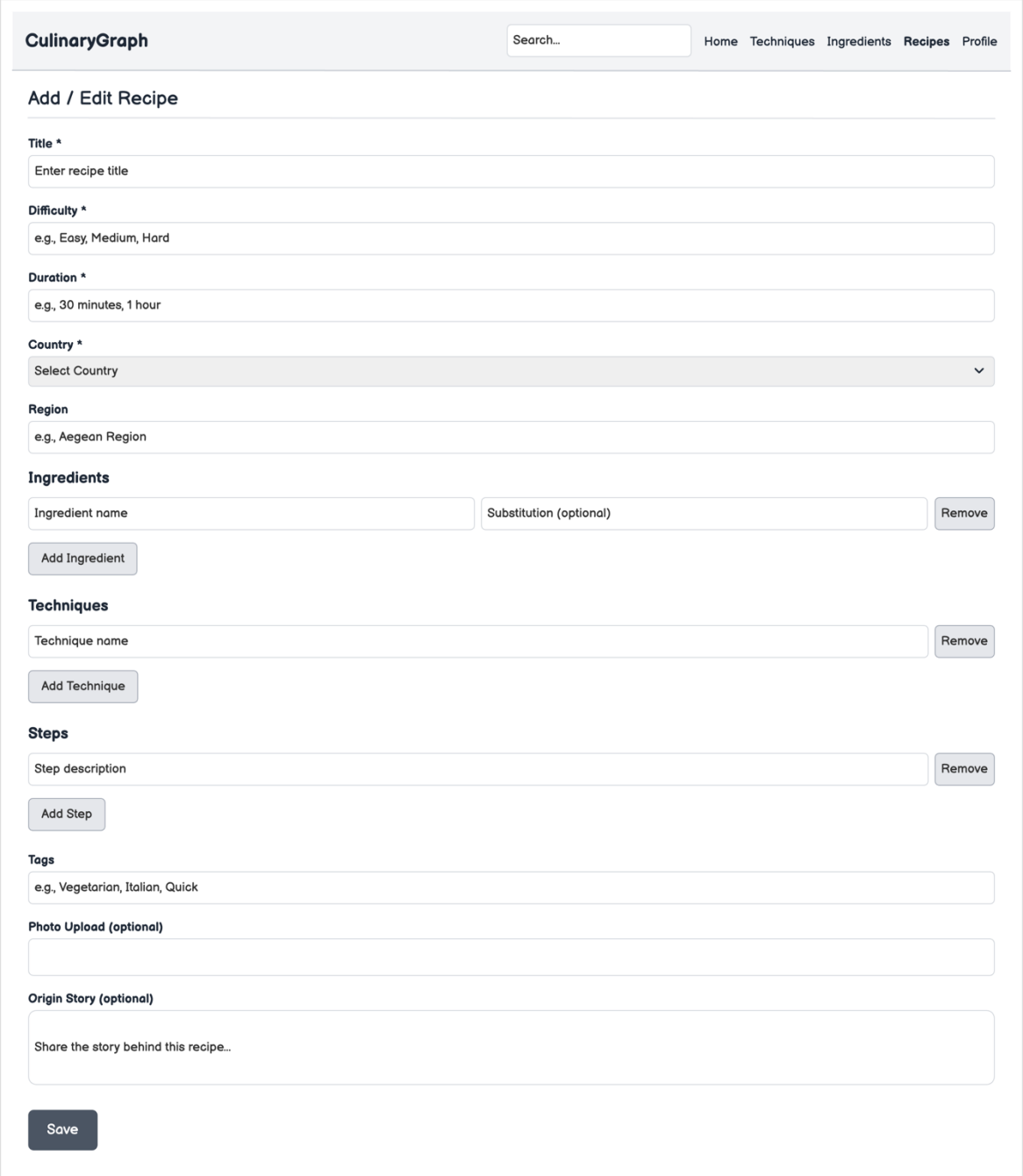

Figure 14: Add Recipe Page

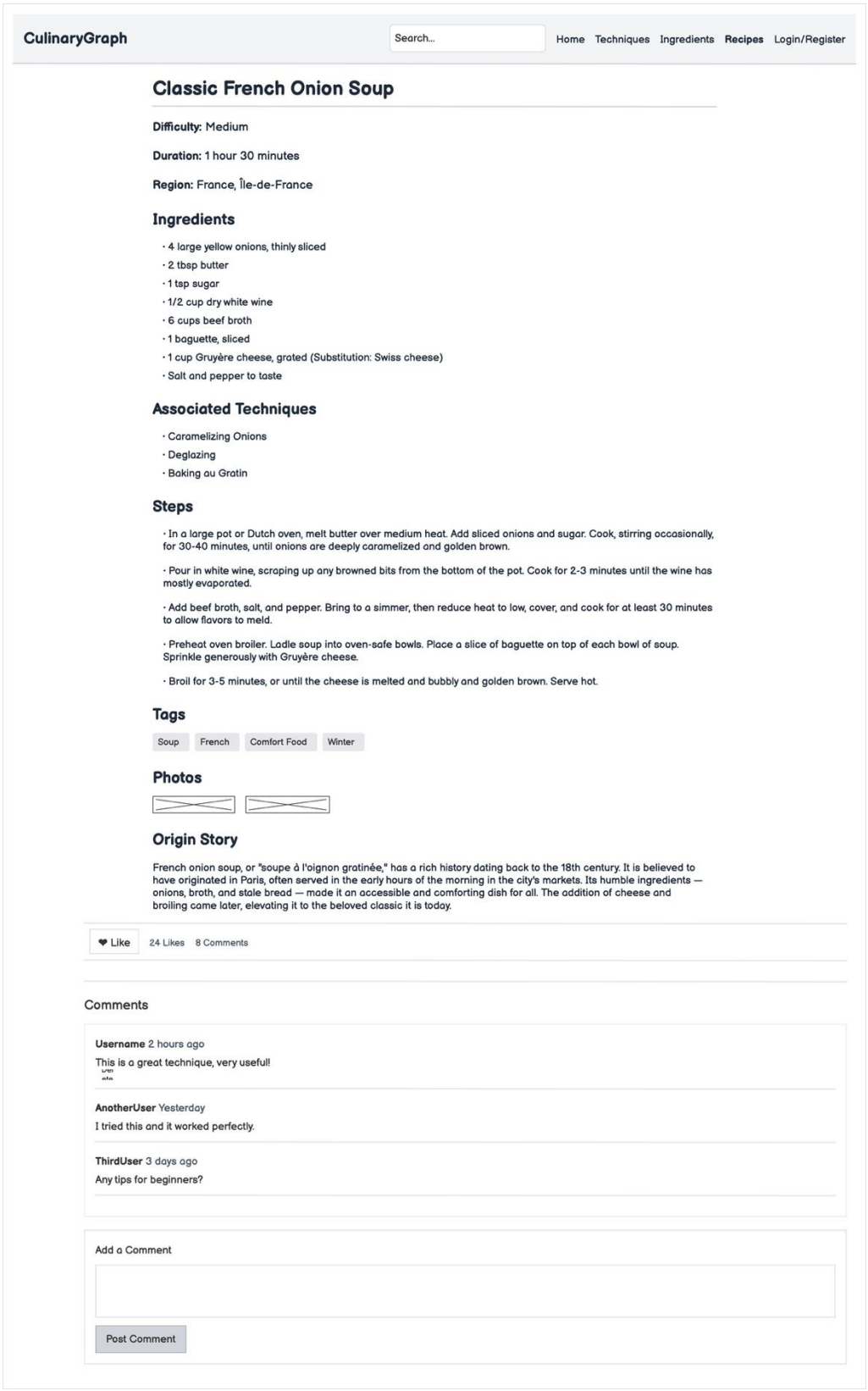

Figure 15: Recipe Detail Page

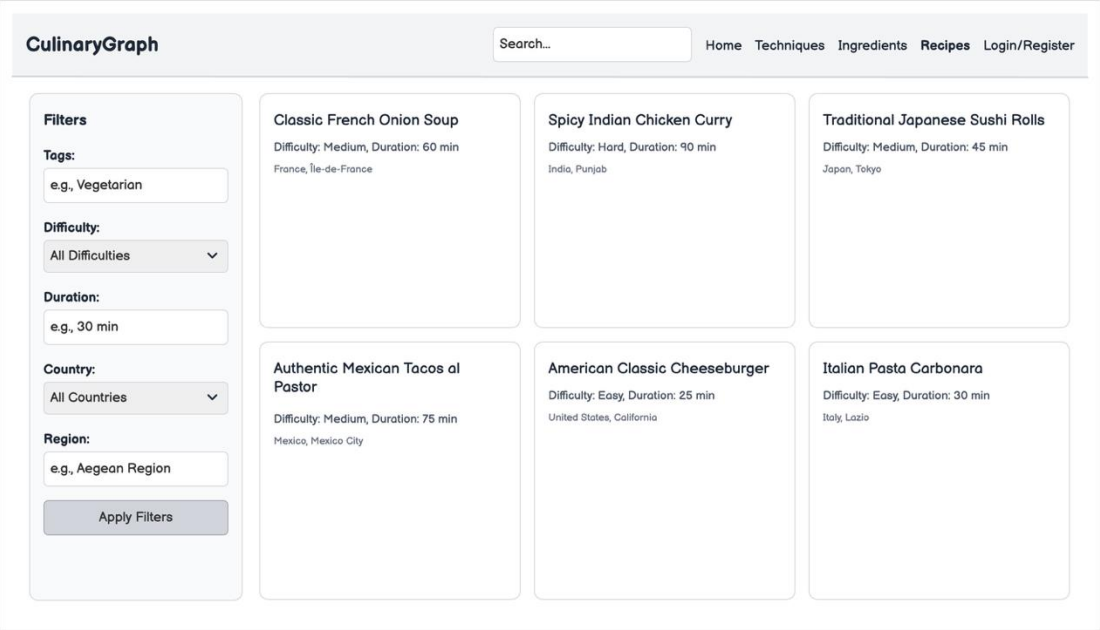

Figure 16: Recipe List Page

6. Status of the Project

The following work breakdown table decomposes the project into discrete work packages, links them to phases and deliverables, and supports planning and status tracking.

ID	Task	Deliverable	Requirements	Status	Deadline
BL-01	Software Requirements Specification	SRS	FR-1–FR-21, NFR-1–NFR-12	Done	2026-02-23
BL-02	User scenarios & personas	User_Scenarios	FR-1–FR-21 (acceptance narratives)	Done	2026-02-23
BL-03	Project plan, communication, RACI	ProjectPlan.md (this file)	Course plan, traceability	Done	2026-02-28
BL-04	Deliverables index & repo hygiene	DELIVERABLES_INDEX	Documentation (SRS §4.1)	Done	2026-03-02
BL-05	Containerized stack (Postgres, Keycloak, app)	docker/	FR-2–FR-3, NFR-7, NFR-9	Done	2026-03-09
BL-06	Guest register / login / logout (Keycloak)	App + Keycloak (PKCE / session)	FR-1, FR-2, FR-3	Done	2026-03-16
BL-07	Technique create/read/update/delete (owner)	Backend + frontend catalog	FR-4, FR-7, FR-17, FR-21	Done	2026-03-23
BL-08	Technique search & filters	List + filter UI	FR-5, NFR-1, NFR-5	Done	2026-03-23
BL-09	Technique detail view	Detail page	FR-6	Done	2026-03-23
BL-10	Ingredient create/read/update/delete (owner)	Backend + frontend catalog	FR-8, FR-11, FR-17, FR-21	Done	2026-03-30
BL-11	Ingredient search & filters	List + filter UI	FR-9, NFR-1, NFR-5	Done	2026-03-30
BL-12	Ingredient detail view	Detail page	FR-10	Done	2026-03-30

BL-13	Recipe create/read/update/delete (owner)	Backend + frontend recipes	FR-12, FR-15, FR-17, FR-21	Done	2026-04-13
BL-14	Recipe search & filters	List + filter UI	FR-13, NFR-1, NFR-5	Done	2026-04-13
BL-15	Recipe detail + links to catalog	Detail + deep links	FR-14, FR-16	Done	2026-04-13
BL-16	Homepage highlighted / recent content	Home view	FR-18	Done	2026-04-20
BL-17	Comments on recipe / technique / ingredient	Social API + UI	FR-19	Done	2026-04-27
BL-18	Likes + aggregate count (per entity)	Social API + UI	FR-20	Done	2026-04-27
BL-19	Contributor identity on list & detail	createdBy + profile link where applicable	FR-21	Done	2026-04-20
BL-20	Server-side ownership enforcement	Services / controllers	FR-3, FR-7, FR-11, FR-15, NFR-8	Done	2026-04-13
BL-21	Knowledge map (geo exploration)	Home / map UX	FR-5, FR-9, FR-13, FR-18 (interpretation)	Done	2026-05-05
BL-23	User manual (end user)	USER_MANUAL	SRS §4.1 documentation	Done	2026-05-06
BL-24	Developer README & API notes	Repo README, MILESTONE_REVIEW	SRS §4.1, NFR-11	Done	2026-05-06
BL-25	Milestone / course review document	MILESTONE_REVIEW	All FR/NFR (status)	Done	2026-05-09
BL-26	Release tag & release notes	RELEASE_NOTES	Delivery	Done	2026-05-10
BL-27	Short video demo	demo.mp4	Submission	Done	2026-05-10
BL-28	Production deploy (e.g. culinary.page)	Hosted deployment	NFR-7, NFR-9	Done	2026-05-10
DG-01	Use case diagram	CulinaryGraph_UseCaseDiagram.drawio	FR-1–FR-21 (actor goals), NFR-3 (UI roles)	Done	2026-03-09
DG-02	Class diagram (persistence / domain)	class_diagram.mermaid, class_diagram.png	FR-4–FR-20 data shape, FR-17 associations, NFR-8 ownership fields, NFR-11 extensibility	Done	2026-03-12
DG-03	Sequence diagram — contributor creates recipe	add-recipe-sequence.png	FR-12, FR-15, FR-17, NFR-8	Done	2026-03-16
DG-04	UI mockups (Balsamiq / PNG)	mock-ups/	FR-3, FR-5–FR-6, FR-9–FR-10, FR-13–FR-14, FR-18, NFR-3–NFR-5	Done	2026-03-16
DG-05	Architecture / stack decision log	Working notes under .cursor/ (not a formal graded deliverable path)	NFR-9–NFR-12	Done	2026-03-20

Figure 17: Status of the project

7. Status of Deployment

The production site is served at **https://culinary.page/**. The backend API is exposed under the same host via the **/api** path (**https://culinary.page/api**), with the reverse proxy

forwarding /api requests to the Spring Boot service (so the browser uses a single origin and avoids cross-origin issues for the API).

The application stack (including database, identity provider, backend, and frontend as defined for deployment) runs in **Docker** (or **Docker Compose**, if that matches your setup) on the server.

Item	Answer
URL	https://culinary.page/
API	https://culinary.page/api
Dockerized	Yes

Figure 18: Short table for deployment status

8. Full Installation Instructions

8.1. System requirements

Resource	Minimum	Recommended
Operating system	Linux (e.g. Ubuntu 22.04 LTS / Debian 12) or macOS for development	Ubuntu 24.04 LTS (production)
CPU	2 cores	4 cores
RAM	4 GB	8 GB
Disk	20 GB free	40 GB
Network (production)	TCP 22 (SSH), 80 (HTTP), 443 (HTTPS)	Same

Notes: Keycloak needs a significant JVM footprint (on the order of ~512 MB heap).
With PostgreSQL (two databases), Keycloak, Spring Boot, and the frontend container, plan for roughly 2.5 GB or more of available RAM while the stack is running.

Software prerequisites

- Git — clone the repository.
- Docker Engine with Docker Compose v2 (docker compose, not legacy docker-compose), or Podman + podman-compose as an alternative (podman compose in place of docker compose).
- Optional (production on a Linux server): host nginx, certbot for TLS (see project deployment guide).

8.2. Full stack with Docker

The repository ships a multi-service definition under docker/docker-compose.yml: PostgreSQL (application database), PostgreSQL (Keycloak database), Keycloak 24, Spring Boot backend, and React SPA behind nginx.

1. Clone the repository

```
git clone https://github.com/yaseminyumak/SWE-573.git
cd SWE-573/docker
```

2. Configure for your environment (first-time)

- Edit docker-compose.yml as needed, in particular frontend build arguments VITE_KEYCLOAK_URL (browser-reachable Keycloak URL, no trailing slash) and VITE_API_BASE (typically /api).
- Change default passwords for postgres, postgres-kc, keycloak, and backend database credentials before any public deployment (see variable table in docker/README.md).

3. Build and start

```
docker compose build
docker compose up -d
```

4. Wait for services

- Follow Keycloak logs until the realm is ready (often **1–3 minutes**):

`docker compose logs -f keycloak`

- Confirm the backend has started:

`docker compose logs -f backend`(look for the Spring Boot “Started …Application” line).

5. Access URLs (default host port mapping in docker-compose.yml)

Service	URL (typical local use)
Web UI (nginx)	http://localhost/ (frontend container maps host 80 → container 80)
Backend API	http://localhost:8080 (also reachable; the UI proxies /api to the backend inside the frontend container)
Keycloak	http://localhost:8180 (admin console and OIDC; in production, Keycloak is often reached only via reverse proxy—see docker/README.md §3.4)
Application PostgreSQL	localhost:5432 (bound on the host in the default compose file—restrict firewall in production)

6. Stop and reset

```
# Stop containers (volumes preserved)
docker compose down
# Stop and remove volumes (full data reset, including DB and Keycloak state)
docker compose down -v
```

7. Useful maintenance commands

```
docker compose logs -f backend
```

```
docker compose build backend && docker compose up -d --force-recreate backen
```

9. API Documentation

9.1. Live documentation links

Resource	Production	Local (Docker Compose nginx)	Local (backend only)
Interactive API UI (Swagger UI)	https://culinary.page/docs	http://localhost/docs	http://localhost:8080/docs
OpenAPI 3 specification	https://culinary.page/openapi.yaml	http://localhost/openapi.yaml	http://localhost:8080/openapi.yaml

9.2. Usage scenarios

Scenario A — Toggle a like on a recipe

- Endpoint: POST /api/social/likes
- Description: An authenticated user toggles a like on a published recipe and receives the updated like count and whether the current user has liked the entity.
- cURL:

```
curl --request POST "${BASE}/api/social/likes" \
 --header "Authorization: Bearer ${ACCESS_TOKEN}" \
 --header "Content-Type: application/json" \
 --data '{
  "entityType": "RECIPE",
  "entityId": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee"
 }'
```

- JSON response:

```
{
 "count": 4,
 "likedByMe": true
}
```

Scenario B — Add a comment on an ingredient

- Endpoint: POST /api/social/comments
- Description: A signed-in user posts a comment on an ingredient profile; the API returns the new comment id, username, body, and creation timestamp.
- cURL:

```
curl --request POST "${BASE}/api/social/comments" \
 --header "Authorization: Bearer ${ACCESS_TOKEN}" \
 --header "Content-Type: application/json" \
 --data '{
  "entityType": "INGREDIENT",
  "entityId": "11111111-2222-3333-4444-555555555555",
  "body": "We substitute jarred tamarind paste when fresh pods are unavailable; add lime for acidity."
 }'
```

- JSON response:

```
{
 "id": "66666666-7777-8888-9999-aaaaaaaaaaaa",
 "username": "emre_researcher",
 "body": "We substitute jarred tamarind paste when fresh pods are unavailable; add lime for acidity.",
 "createdAt": "2026-05-10T14:32:01.123456789Z"
}
```

Scenario C — Create a recipe with catalog associations

- Endpoint: POST/api/recipes
- Description: A contributor creates a recipe with ordered steps, ingredient lines (optional catalog ingredientId), and associatedTechniqueIds, exercising structured validation beyond a simple read.
- cURL:

```
curl --request POST "${BASE}/api/recipes" \
 --header "Authorization: Bearer ${ACCESS_TOKEN}" \
 --header "Content-Type: application/json" \
 --data '{
  "title": "Regional şakşuka with charred peppers",
  "description": "Weeknight-friendly variant using broiler char for depth.",
  "difficulty": "MEDIUM",
  "durationMinutes": 40,
  "country": "Turkey",
  "tags": ["vegetarian", "mezze"],
  "originStory": "Adapted from family notes and catalog technique references.",
  "steps": [
   { "order": 1, "instruction": "Char peppers under broiler; peel and strip." },
   { "order": 2, "instruction": "Fry eggplant coins until golden; drain." },
   { "order": 3, "instruction": "Simmer tomatoes with garlic; fold vegetables; finish with parsley." }
  ],
  "ingredients": [
```

```
    {
      "name": "Eggplant",
      "quantity": "2",
      "unit": "pcs",
      "ingredientId": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee"
    },
    { "name": "Olive oil", "quantity": "3", "unit": "tbsp" }
  ],
  "associatedTechniqueIds": [
    "bbbbbbbb-cccc-dddd-eeee-ffffffffffff"
  ]
}'
```

- JSON response (truncated; structure as returned by the API):


```
{
  "id": "cccccccc-dddd-eeee-ffff-000000000001",
  "title": "Regional şakşuka with charred peppers",
  "description": "Weeknight-friendly variant using broiler char for depth.",
  "difficulty": "MEDIUM",
  "durationMinutes": 40,
  "status": "PUBLISHED",
  "createdBy": "defne_cook",
  "createdAt": "2026-05-10T15:05:22.123456789Z",
  "updatedAt": "2026-05-10T15:05:22.123456789Z",
  "steps": [
    { "order": 1, "instruction": "Char peppers under broiler; peel and strip." },
    { "order": 2, "instruction": "Fry eggplant coins until golden; drain." },
    { "order": 3, "instruction": "Simmer tomatoes with garlic; fold vegetables; finish with parsley." }
  ],
  "ingredients": [
    {
      "name": "Eggplant",
      "quantity": "2",
      "unit": "pcs",
      "ingredientId": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee"
    },
    {
      "name": "Olive oil",
      "quantity": "3",
      "unit": "tbsp",
      "ingredientId": null
    }
  ],
```

```
 "country": "Turkey",
 "tags": ["vegetarian", "mezze"],
 "originStory": "Adapted from family notes and catalog technique references.",
 "associatedTechniqueIds": [
  "bbbbbbbb-cccc-dddd-eeee-ffffffffffff"
 ]
}
```


10. User Manual

This guide explains how to use the Culinary Graph web application in clear, beginner-friendly language.

10.1. Introduction

10.1.1. Project overview

CulinaryGraph is a web application for region-aware culinary knowledge. It brings together recipes, cooking techniques, and ingredient profiles so you can read and share structured information—not only steps, but also country, region, cultural notes, substitutions, and stories.

10.1.2. Purpose of the platform

- Help learners and curious cooks discover how dishes and methods connect to places and culture.
- Let contributors add and maintain their own techniques, ingredients, and recipes with consistent fields and optional photos and links between catalog entries.

10.1.3. Supported user roles

Role	What you can do
Guest (not signed in)	Browse the Home page, open Techniques, Ingredients, and Recipes lists and detail pages, use search (header) and list filters where available. You cannot create or edit content.
Contributor (signed in)	Everything a Guest can do, plus create, edit, and delete your own techniques, ingredients, and recipes; upload images on entities you own; manage your entries from Profile.

Figure 19: User Roles

Note: The application uses Keycloak for accounts. Advanced admin-only tools (if any) are outside this manual's scope.

10.2. Authentication

Identity is handled by Keycloak. You sign in and register through the Login / Register flow in the app header.

10.2.1. Registration

1. Open CulinaryGraph in your browser at https://culinary.page/.
2. Click Login / Register in the top bar.
3. Choose Register and complete the sign-up form (for example username, email, and password, as required).
4. After successful registration, you may be signed in automatically or asked to sign in—follow the on-screen prompts.

10.2.2. Login

1. Click Login / Register.
2. Enter your username and password on the sign-in form.
3. You are returned to CulinaryGraph as a signed-in user. The top bar should show Profile and Logout instead of Login / Register.

10.2.3. Logout

1. While signed in, click Logout in the top bar.
2. Your session ends and you return to anonymous browsing.

10.3. Browsing & discovery

10.3.1. Browse recipes, techniques, and ingredients

Use the main navigation (top of every page):

Link	What you see
Home (/)	Knowledge Map (see §3.4), Highlighted and Recently Added content.
Techniques (/catalog/techniques)	List of techniques with filters (see §3.3). Open an item to see its detail page.
Ingredients (/catalog/ingredients)	List of ingredient profiles with filters. Open an item for detail.
Recipes (/recipes)	List of recipes with filters. Open an item for detail.

Figure 20: Browsing Results

10.3.2. Homepage search

1. Use the search box in the header (available on all pages).
2. Type a keyword and press Enter.

3. You are taken to the Search page with results grouped into Recipes, Techniques, and Ingredients whose titles or names contain your query (simple text match).
4. Click a result row to open that entity's detail page.

If there are no matches, try another keyword.

10.3.3. List filters (techniques, ingredients, recipes)

On Techniques, Ingredients, and Recipes list pages, use the filter controls (sidebar or panel) to narrow by options such as:

- Country
- Region (text filter where shown)
- Season (ingredients)
- Difficulty, tags, and other recipe filters (as shown on the page)

Use Clear all filters (or equivalent) to reset. Only items matching the filters appear in the list.

10.3.4. Knowledge Map

The Knowledge Map on the Home page is rolling out: it gives you a geographic view of what the community has added.

- For each country on the map, you can see what content exists for that country (recipes, techniques, and ingredients that contributors have linked to that place).
- When you choose Explore (or the equivalent action) for a country, the app shows a single list of all content entered for that country—so you can scan everything in one place before opening a detail page.

10.3.5. Navigate between related content

- From a recipe detail page, use Edit (if you own the recipe), associated techniques, and any links shown to move to related technique or ingredient pages where the UI provides them.
- From technique or ingredient detail pages, use related ingredients / related techniques when present.
- Use the top navigation to switch between major areas at any time.

10.4. Recipes

You must be signed in to create, edit, or delete recipes.

10.4.1. Create a recipe

1. Go Recipes → + Add Recipe (or open /recipes/new).

2. Fill in Title, Difficulty, Duration (optional), Country, Region (if shown), Steps, and Ingredient lines (name; optional quantity, unit, substitution).
3. Optionally add tags (comma-separated), origin story, and select associated techniques from the catalog picker.
4. Submit the form to save. You are redirected to the recipe list or detail view.

10.4.2. Edit a recipe

1. Open your recipe's detail page.
2. If you are the owner, click Edit (/recipes/:id/edit).
3. Change fields and save.

10.4.3. Delete a recipe

On the recipe detail page (as owner), click Delete and confirm or use Profile → My Recipes → Delete.

10.4.4. Ingredients, techniques, steps, and regions on a recipe

Part	How
Steps	Add multiple step lines; they are stored in order.
Ingredients	Add rows with name; optionally quantity, unit, substitution text.
Techniques	Pick associated techniques from the catalog (links the recipe to existing techniques).
Region / country	Use Country and Region fields so readers see geographic context.

Figure 21: Adding Content

10.5. Techniques & ingredients

You must be signed in to create, edit, or delete catalog entries you own.

10.5.1. Content creation flow

New technique

1. Techniques → + Add Technique (/catalog/techniques/new).
2. Enter Name, Difficulty, Country, Region, Cultural notes, Prerequisites (optional), Steps, optional photos, and related ingredients where the form offers pickers.
3. Submit to save.

New ingredient

1. Ingredients → + Add Ingredient (/catalog/ingredients/new).
2. Enter Name, Country, Region, Seasons, Provenance story, Substitutions, and optional related techniques.

3. Submit to save.

10.5.2. Editing and deleting owned content

Open a detail page you own and use Edit / Delete, or use Profile → My Techniques / My Ingredients. You cannot change other users' entries from the UI.

10.5.3. Linking related entities

- Technique form: related ingredients (catalog).
- Ingredient form: related techniques (catalog).
- Recipe form: associated techniques (catalog).

These links help readers jump between related detail pages.

10.6. Social features

10.6.1. Likes

On recipe, technique, and ingredient detail pages, use the Like control to show appreciation for content you enjoy. You can remove your like if you change your mind. The page shows an aggregate like count so everyone can see how often an entry has been liked.

10.6.2. Comments

Open a recipe, technique, or ingredient you want to discuss. Use the Comments area to read what others wrote and to add your own comment when you are signed in. Keep comments respectful and on-topic; empty or invalid submissions are rejected.

10.6.3. Viewing contributor information

On recipe, technique, and ingredient detail pages, look for Created by to see who contributed the entry (from the identity system).

10.7. Profile management

Open Profile when signed in.

10.7.1. Viewing your own content

Profile lists My Techniques, My Ingredients, and My Recipes with Edit and Delete for each row you own.

10.7.2. Activity history

There is no separate chronological activity feed. Profile is the main place to see everything you have created.

10.7.3. Managing uploaded images

On detail pages you own, use the image manager to upload, reorder, or remove images for that entity (recipes, techniques, ingredients where shown). Guests can view images; only owners get upload/delete controls.

11. A Use Case

- Name: Contributor publishes a recipe; another user discovers, inspects authorship, and engages
- Scope: CulinaryGraph web application (production: https://culinary.page/), backend under https://culinary.page/api, identity (Keycloak), persistence (database).
- Level: User goal (sea-level): completes authoring, publication to discovery surfaces, and reader-side interaction in one continuous demonstration.
- Goal: Show a complete vertical slice: authenticated contributor creates rich recipe content; the system validates and persists it; the recipe becomes discoverable; a second person finds it, inspects who created it, and records social feedback (like, comment)—including the underlying client–server exchanges through /api.
- Stakeholders & interests:
    - Contributor: Reliable save, clear errors if invalid, visible attribution.
    - Reader: Easy find, readable detail, trustworthy author link, quick feedback actions.
    - Operator / course reviewer: Evidence of auth, validation, persistence, search/list integration, and engagement features.
- Primary actors
    - Contributor (authenticated).
    - Reader (another person; may be guest for view/search, or must be signed in for like/comment—align with your deployed rules).
- Secondary actors / systems
    - Identity provider (Keycloak): login, tokens.
    - Reverse proxy: routes / to frontend, /api to backend.
    - API (Spring Boot): validation, authorization, business rules, persistence.
- Database: stores recipe, user references, likes, comments as implemented.
- Preconditions: Contributor account exists and is permitted to create recipes.

Reader can reach the site; search/browse indexes include newly created recipes after save (or within normal refresh/cache behavior—state honestly if there is delay).
Browser supports SPA navigation; cookies/storage allow session as configured.

- Trigger: Contributor chooses to add a new recipe after signing in.
- Main success scenario
    - Authoring (contributor)

Reader (later step) is not required yet; contributor navigates to https://culinary.page/.
Contributor selects Login; the application redirects to Keycloak; contributor authenticates; the application returns with an active session and shows authenticated UI (e.g. profile / logout).
Contributor opens Create recipe (or equivalent route).
Contributor enters title and any required metadata your form enforces (e.g. difficulty, duration if mandatory).
Contributor adds ingredient lines (names; quantities/units/substitution if offered).
Contributor associates techniques (picker, tags, or links—match your UI).
Contributor sets region information (country, region, and any other location fields on the form).
Contributor enters preparation steps (ordered steps as required).
Contributor optionally adds images or other optional fields if part of your demo script.
Contributor submits the form.
Frontend sends a create recipe request to https://culinary.page/api/... with JSON body and Bearer access token (or equivalent auth header your app uses).
Reverse proxy forwards /api to the backend.
Backend authenticates the token, authorizes the operation, validates payload (required fields, lengths, references to catalog IDs if applicable).
On success, backend persists the recipe (and related rows: ingredients lines, links, images metadata, etc., as implemented).
Backend responds with success (e.g. 201 Created / 200 OK) and identifying data (e.g. recipe id).
Frontend shows a success state and navigates to recipe detail or edit view as designed, so the contributor can confirm content.

- Publication & discovery

Contributor logs out or leaves the session open—either is fine for the demo narrative.
Reader opens https://culinary.page/ (same or different browser/profile).
Reader uses search (global search or recipe list filters—match your UI) with a keyword from the recipe title or region.
Search results include the new recipe; reader selects it.

- Consumption & engagement

Application loads recipe detail: shows title, steps, ingredients, techniques, region, images if any, and author/contributor attribution.
Reader follows author profile (link from recipe or “Created by”) and verifies contributor identity information shown by the app.
Reader returns to the recipe detail page.
Reader activates Like; UI updates count; request hits /api and persists like (if guest likes are disabled, reader signs in first—then repeat like).
Reader enters comment text and submits; UI shows the new comment in the thread after successful /api round-trip.

- Postconditions:

Recipe row (and dependent data) exists in the database and is retrievable by id and by search/list endpoints.
Reader session shows like state and new comment consistent with server data.
Contributor profile reflects ownership / listing of created content as your profile page defines.

12. Test Results

12.1. User Test Scenarios

Scope	ID	Preconditions	Steps	Expected	Status	SRS
Authentication and session	UT-AUTH-01	Guest	Open app; use Login / Register; complete Register with valid data.	Account is created per Keycloak flow; user ends signed in or is prompted to sign in successfully.	success	FR-1, FR-2
Authentication and session	UT-AUTH-02	Guest	Open Login / Register; sign in with valid Contributor credentials.	Session established; header shows Profile and Logout.	success	FR-2
Authentication and session	UT-AUTH-03	Signed in	Click Logout.	Session ends; Login / Register visible again; protected create/edit actions not available as Guest.	success	FR-2, FR-3
Guest: browse, search, filters	UT-NAV-01	Guest	From header, open Home, Techniques, Ingredients, Recipes in turn.	Each route loads without error; list or home content visible.	success	FR-3, FR-18
Guest: browse, search, filters	UT-NAV-02	Guest	On Home, confirm highlighted and/or recently added sections show entries when seed data exists.	Sections render expected entities or empty state is clear.	success	FR-18
Guest: browse, search, filters	UT-SEA-01	Guest	Use header search; enter a keyword known to exist in a title/name; submit.	Search page shows grouped results (recipes, techniques, ingredients) or empty state; clicking a row opens detail.	success	FR-3, FR-5, FR-9, FR-13
Guest: browse, search, filters	UT-FLT-01	Guest	Open Techniques; apply country and/or region (and text if present); observe list.	List only shows matching items; clear filters resets the list.	success	FR-5, NFR-5
Guest: browse, search, filters	UT-FLT-02	Guest	Open Ingredients; apply season and/or region/country filters.	List reflects filters; clear works.	success	FR-9, NFR-5
Guest: browse, search, filters	UT-FLT-03	Guest	Open Recipes; apply difficulty, tags, or other shown filters.	List reflects filters; clear works.	success	FR-13, NFR-5
Detail views and cross-navigation	UT-DTL-01	Guest	Open a technique that has related ingredients recorded.	Detail shows stored fields (name, steps, country, region, notes, prerequisites, photos if any) and related ingredients where linked.	success	FR-6

Detail views and cross-navigation	UT-DTL-02	Guest	Open an ingredient that has related techniques.	Detail shows ingredient fields and related techniques where linked.	success	FR-10
Detail views and cross-navigation	UT-DTL-03	Guest	Open a recipe with steps, ingredient lines, tags, origin story, photos if any.	All provided fields visible per recipe detail spec.	success	FR-14
Detail views and cross-navigation	UT-LNK-01	Guest	From a recipe detail, select a linked technique or linked catalog ingredient (when UI exposes links).	Browser navigates to the correct technique or ingredient detail page.	success	FR-16
Detail views and cross-navigation	UT-ATT-01	Guest	Open list and detail for a technique, ingredient, and recipe that expose creator identity.	Authorship (or contributor display name) visible where backend provides it.	success	FR-21
Contributor: recipes	UT-REC-01	Contributor	Recipes → Add recipe; fill required fields (title, difficulty, steps, ingredient lines); optionally tags, origin story, associated techniques; submit.	Recipe persists; redirect to list or detail; new row visible.	success	FR-12, FR-17
Contributor: recipes	UT-REC-02	Contributor, owns recipe	Open own recipe detail → Edit; change fields; save.	Updates persist on reload.	success	FR-15
Contributor: recipes	UT-REC-03	Contributor, owns recipe	Delete own recipe (if UI offers delete).	Entry removed or clearly archived per product behavior.	success	FR-15
Contributor: recipes	UT-REC-04	Contributor	Open another user's recipe detail.	Edit/delete for that entity are not offered, or server rejects unauthorized actions.	success	FR-15, NFR-8
Contributor: techniques	UT-TEC-01	Contributor	Add technique with name, steps, country, region, cultural notes; optional photos, prerequisites, related ingredients from picker; submit.	Technique appears in list and detail with associations where supported.	success	FR-4, FR-17
Contributor: techniques	UT-TEC-02	Contributor, owns technique	Edit and save own technique.	Changes persist.	success	FR-7
Contributor: techniques	UT-TEC-03	Contributor	Attempt to edit/delete a technique owned by another user (via URL or UI if exposed).	Operation denied consistently (UI and/or API error).	success	FR-7, NFR-8
Contributor: ingredients	UT-ING-01	Contributor	Add ingredient with name, country, region(s), seasonality, sourcing notes, substitutions; optional related techniques; submit.	Profile appears in list and detail.	success	FR-8, FR-17
Contributor: ingredients	UT-ING-02	Contributor, owns profile	Edit and save own ingredient.	Changes persist.	success	FR-11
Contributor: ingredients	UT-ING-03	Contributor	Attempt to edit/delete another user's ingredient profile.	Operation denied.	success	FR-11, NFR-8
Profile and ownership	UT-PRF-01	Contributor	Open Profile (or equivalent).	Lists or summarizes own techniques, ingredients, and recipes for management per user manual.	success	FR-3, FR-7, FR-11, FR-15
Homepage Knowledge Map	UT-MAP-01	Guest	On Home, interact with Knowledge Map.	The user can select a country on the map, choose Explore, and see content for that country.	success	

12.2. Unit Testing

- Test scope and layers

Layer	Location (examples)	What is verified
Domain	TechniqueTest, IngredientTest, RecipeTest	Entity creation rules, trimming, defaults, publish/archive transitions, immutability of exposed collections
Application	TechniqueServiceTest, IngredientServiceTest, RecipeServiceTest	CRUD orchestration, not-found errors, null-safe handling (e.g. empty associated techniques / seasons)
API	TechniqueControllerTest, IngredientControllerTest, RecipeControllerTest	HTTP status codes (200, 201, 400, 404), JSON payloads, validation on invalid input
Frontend	TechniqueListPage.test.tsx, RecipeFormPage.test.tsx, RelationPicker.test.tsx, useCatalogIndex.test.ts	List filters, form steps, shared components and hooks

- Representative unit tests

#	Component / class	Test method (illustrative)	Expected behavior
1	RecipeTest	create_throwsWhenTitleIsNull	Recipe cannot be constructed without a title
2	RecipeServiceTest	getById_throwsWhenNotFound	Service throws when ID does not exist
3	RecipeServiceTest	create_withNullAssociatedTechniques_usesEmptySet	Null optional associations do not break creation
4	RecipeControllerTest	create_returns400WhenTitleIsBlank	API returns 400 for invalid body
5	RecipeControllerTest	getById_returns404WhenNotFound	API returns 404 for missing resource
6	TechniqueListPage (Vitest)	filters by country after clicking Apply Filters	UI applies country filter and updates displayed list

- Aggregated results

Suite	Command	Tests run	Passed	Failed	Skipped	Duration
Backend unit tests	./mvnw test (from culinarygraph-backend)	107	107	0	0	3.95s
Frontend unit tests	npx vitest run (from culinarygraph-frontend)	66	66	0	0	1.26s